

Міністерство освіти і науки України
Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Звіт
з лабораторної роботи No 3
з дисципліни
«Мова програмування Java»

Виконала:
студентка групи ЗПІ-зп41
Ірина Левківська

Перевірив:
Орленко С.П.

Київ 2025

Поставка завдання (Завдання 3.1)

Напишіть консольний додаток, використовуючи архітектурний шаблон MVC, який:

- описує інтерфейс Drawable з методом побудови фігури draw(); описує абстрактний клас Shape, який реалізує інтерфейс Drawable і містить поле shapeColor типу String для кольору фігури і конструктор для його ініціалізації, абстрактний метод обчислення площі фігури calcArea() і перевизначений метод toString();

- описує класи Rectangle, Triangle, Circle, які успадковуються від класу Shape і реалізують метод calcArea (), а також перевизначають метод toString (); створює набір даних типу Shape (масив розмірністю не менш 10 елементів);

- обробляє масив:

- відображає набір даних;
- обчислює сумарну площу всіх фігур набору даних;
- обчислює сумарну площу фігур заданого виду;
- впорядковує набір даних щодо збільшення площі фігур, використовуючи об'єкт інтерфейсу Comparator;
- впорядковує набір даних за кольором фігур, використовуючи об'єкт інтерфейсу Comparator.

Значення для ініціалізації об'єктів вибираються з заздалегідь підготовлених даних (обраних випадковим чином або по порядку проходження).

Вирішення завдання

При розробці додатка я обрала архітектурний шаблон MVC, щоб розділити дані, логіку обробки та виведення результатів. Це дозволило зробити код більш структурованим і легким для розширення.

Модель (Model): Основою став абстрактний клас Shape. Я вирішила зробити його абстрактним, оскільки саме поняття "фігура" занадто загальне, щоб мати конкретну площу. Проте він задає загальне поле для всіх фігур — колір (shapeColor). Також я імплементував інтерфейс Drawable. У кожного конкретного класу (я створив Rectangle, Triangle та Circle) я перевизначив метод calcArea(), використовуючи відповідні математичні формули, та метод toString() для зручного виводу даних.

Вигляд (View): Для відображення інформації я створив окремий клас ShapeView. Його завдання максимально просте — отримувати дані від

контролера та красиво друкувати їх у консоль. Це зручно, бо якщо ми захочемо змінити формат таблиці або виводу, нам не доведеться чіпати логіку обчислень.

Контролер (Controller): Клас ShapeController став "мозком" програми. При ініціалізації він автоматично генерує масив із 10 випадкових фігур. Я додав туди логіку обчислення сумарної площі та фільтрацію за типом (наприклад, для пошуку суми площ тільки кіл). Окрему увагу я приділив сортуванню. Замість того, щоб писати алгоритми сортування вручну, я використав стандартний метод Arrays.sort разом із об'єктами Comparator. Це дозволило мені в декілька рядків реалізувати сортування спочатку за зростанням площі, а потім за назвою кольору в алфавітному порядку.

```
interface Drawable {
    void draw();
}

abstract class Shape implements Drawable {
    protected String shapeColor;
    public Shape(String shapeColor) { this.shapeColor = shapeColor; }
    public abstract double calcArea();
    @Override
    public String toString() {
        return String.format("Колір: %-8s | Площа: %-8.2f", shapeColor, calcArea());
    }
}

class Rectangle extends Shape {
    private double w, h;
    public Rectangle(String c, double w, double h) { super(c); this.w = w; this.h = h; }
    @Override public double calcArea() { return w * h; }
    @Override public void draw() { System.out.println("Draw Rectangle"); }
    @Override public String toString() { return "Rectangle | " + super.toString(); }
}

class Triangle extends Shape {
    private double b, h;
    public Triangle(String c, double b, double h) { super(c); this.b = b; this.h = h; }
    @Override public double calcArea() { return 0.5 * b * h; }
    @Override public void draw() { System.out.println("Draw Triangle"); }
    @Override public String toString() { return "Triangle | " + super.toString(); }
}

class Circle extends Shape {
    private double r;
    public Circle(String c, double r) { super(c); this.r = r; }
    @Override public double calcArea() { return Math.PI * r * r; }
    @Override public void draw() { System.out.println("Draw Circle"); }
    @Override public String toString() { return "Circle | " + super.toString(); }
}
```

```

    public void run() {
        view.printMessage("Дані");
        view.printShapes(shapes);

        double total = 0;
        for(Shape s : shapes) total += s.calcArea();
        view.printValue("Загальна площа", total);

        view.printMessage("Сортування за площею");
        Arrays.sort(shapes, Comparator.comparingDouble(Shape::calcArea));
        view.printShapes(shapes);
    }
}

// 4. ГОЛОВНИЙ КЛАС (Назва файлу повинна бути MVCShapesApp.java)
public class MVCShapesApp {
    public static void main(String[] args) {
        ShapeView view = new ShapeView();
        ShapeController controller = new ShapeController(view);
        controller.run();
    }
}

```

```

class ShapeView {
    public void printShapes(Shape[] shapes) {
        for (Shape s : shapes) System.out.println(s);
    }

    public void printMessage(String m) { System.out.println("\n--- " + m + " ---"); }
    public void printValue(String l, double v) { System.out.printf("%s: %.2f\n", l, v); }
}

// 3. CONTROLLER
class ShapeController {
    private Shape[] shapes;
    private ShapeView view;

    public ShapeController(ShapeView view) {
        this.view = view;
        this.shapes = generateData();
    }

    private Shape[] generateData() {
        String[] colors = {"Red", "Green", "Blue", "Yellow"};
        Random r = new Random();
        Shape[] data = new Shape[10];
        for (int i = 0; i < 10; i++) {
            int type = r.nextInt(3);
            String col = colors[r.nextInt(colors.length)];
            if (type == 0) data[i] = new Rectangle(col, r.nextDouble()*10, r.nextDouble()*10);
            else if (type == 1) data[i] = new Triangle(col, r.nextDouble()*10, r.nextDouble()*10);
            else data[i] = new Circle(col, r.nextDouble()*10);
        }
        return data;
    }
}

```

Скріншот виконання коду:

--- Дані ---

Triangle	Колір: Blue	Площа: 20.44
Triangle	Колір: Blue	Площа: 44.55
Rectangle	Колір: Red	Площа: 7.61
Rectangle	Колір: Blue	Площа: 23.13
Triangle	Колір: Red	Площа: 13.83
Rectangle	Колір: Green	Площа: 53.94
Triangle	Колір: Green	Площа: 8.90
Triangle	Колір: Yellow	Площа: 27.45
Rectangle	Колір: Green	Площа: 0.07
Circle	Колір: Red	Площа: 134.08
Загальна площа: 333.99		

--- Сортуння за площею ---

Rectangle	Колір: Green	Площа: 0.07
Rectangle	Колір: Red	Площа: 7.61
Triangle	Колір: Green	Площа: 8.90
Triangle	Колір: Red	Площа: 13.83
Triangle	Колір: Blue	Площа: 20.44
Rectangle	Колір: Blue	Площа: 23.13
Triangle	Колір: Yellow	Площа: 27.45
Triangle	Колір: Blue	Площа: 44.55
Rectangle	Колір: Green	Площа: 53.94
Circle	Колір: Red	Площа: 134.08

Висновки

У ході роботи було реалізовано додаток з чітким розділенням обов'язків між класами згідно з шаблоном MVC. Використання поліморфізму дозволило обробляти різні типи фігур уніфіковано через базовий клас Shape. Застосування інтерфейсу Comparator продемонструвало ефективний спосіб впорядкування даних без порушення принципу інкапсуляції.